```
iex> [] = [1]
** (MatchError) no match of right hand side value: [1]

iex> %{} = %{a: 1}
%{a: 1}
```

The left side of the list match is empty, and it doesn't match the right side list. The left side of the map match is empty, but it still matches the right side map. Maps are loosely matched, but lists are strictly matched.

Let's see how we can reference existing values in a pattern match clause.

Pinned Values

You are not limited to only assigning variables in a pattern match. The pin operator lets you use the value of an existing variable inside of your pattern match. This is most useful in test suites, where you want to guarantee that different values match inside of a data structure.

Prepend the variable with the ^ symbol to pin its value. Let's see this in action:

```
iex> var = :match
iex> ^var = :match
:match

iex> ^var = :no_match
** (MatchError) no match of right hand side value: :no_match

iex> [^var, second] = [:match, :other]
iex> second
:other
```

Pinned values are strictly enforced, so the result must perfectly match or you'll receive an error. This is intuitive for most data types, but be careful with maps:

```
iex> map = %{a: 1}
iex> ^map = %{a: 1}
%{a: 1}

iex> ^map = %{a: 1, b: 2}
** (MatchError) no match of right hand side value: %{a: 1, b: 2}
```

The map didn't exactly equal the pinned value, so a MatchError was thrown.

You might be wondering what happens if you use a variable twice on the left side of a match. This isn't considered a pinned value, but it behaves similarly.

```
iex> {x, x} = {1, 1}
iex> x
1

iex> {x, x} = {1, 2}
** (MatchError) no match of right hand side value: {1, 2}
```